CO Assignment 3

Cache Demonstrator

Name: Arnav Singhal
MIS: 612203012
SY B. Tech. Comp. Div 1

Problem Statement:

Download and install open-source cache simulator or demonstrator. Explain and demonstrate the following:

1) Associativity demonstration and results – Set Associative Caches
2) Associativity demonstration and results – Direct Mapped Caches
3) Block replacement policy FIFO
4) Block replacement policy LRU

INTRODUCTION

What is Cache?

Cache is a small, fast memory located close to the CPU that stores copies of frequently accessed data from main memory to reduce access time. Caches work by exploiting the principles of temporal and spatial locality.

There are typically three levels of cache:

- **L1 Cache (Level 1):** The smallest and fastest cache located within the CPU core. It is usually split into instruction and data caches.
- **L2 Cache (Level 2):** Larger than L1, slower, but still faster than main memory. It may be shared between cores or be private per core.
- **L3 Cache (Level 3):** Even larger and slower, typically shared among all CPU cores. It acts as the last-level cache before accessing main memory.

Cache Mapping Techniques

1. <u>Direct Mapped Cache:</u>
   In a direct mapped cache, each memory block is mapped to exactly one location in the cache. This is achieved by using a portion of the memory address (typically the index bits) to determine the cache line where the block will be placed. While it is simple and fast, it can suffer from high conflict misses if multiple blocks map to the same cache line.
2. <u>Associative Cache (Fully Associative):</u>
   In a fully associative cache, any memory block can be placed in any cache line. There is no fixed mapping. This flexibility reduces conflict misses but increases the complexity of the hardware, since each cache line must be searched simultaneously to find a match. Tag comparison is done with all entries in parallel.
3. <u>Set Associative Cache:</u>
   Set associative caching strikes a balance between direct mapped and fully associative caching. The cache is divided into sets, and each set contains multiple lines (ways). A block maps to a specific set using index bits from the address, and within that set, it can occupy any of the lines. For example, in a 4-way set associative cache, each set has four cache lines.

Block Replacement

When a new block needs to be loaded into a cache set that is already full (i.e., all lines are occupied), the cache must choose one existing block to evict. This process is called **block replacement**. It is essential to ensure that new data can be accommodated when there is no free space.

There are various approaches for block replacement:

- **FIFO (First In, First Out):** The oldest block in the set is replaced first.
- **LRU (Least Recently Used):** Replaces the block that has not been used for the longest time.
- **Random Replacement:** A block is chosen at random to be replaced.
- **LFU (Least Frequently Used):** Replaces the block accessed the least number of times.

1) Associativity Demonstration - Set Associative Caches

In a set associative cache,

Address Bit Partitioning

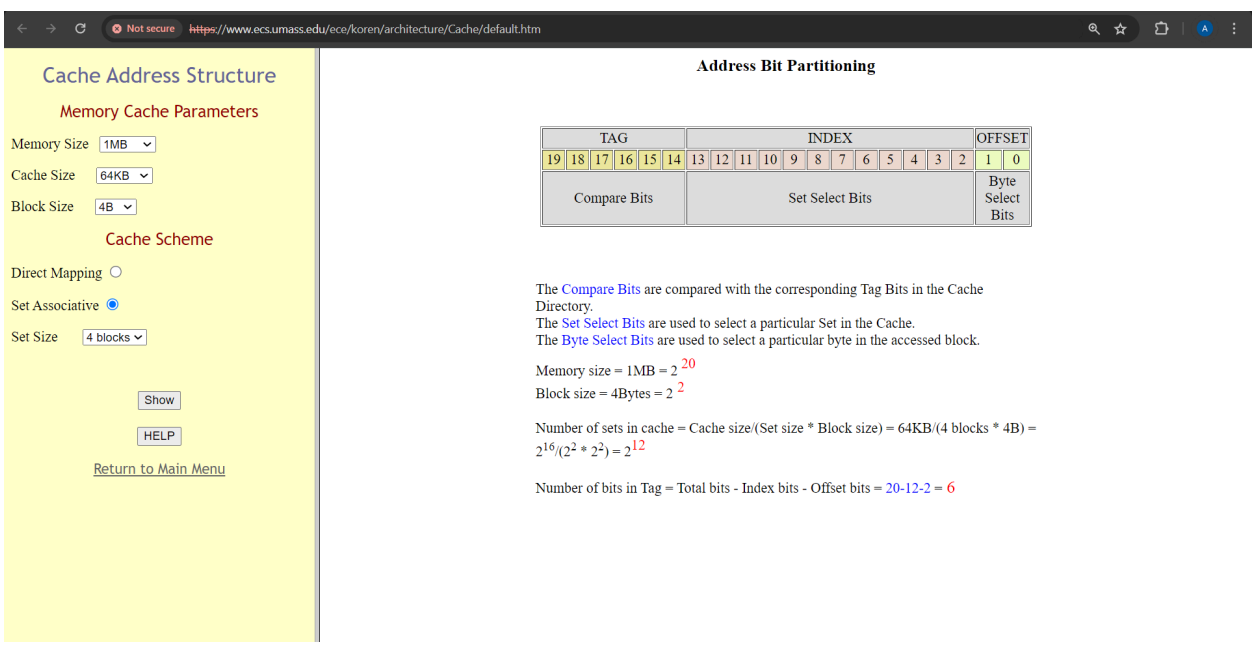
To access data from cache, the memory address is divided into three parts:

- **Tag:** Used to identify if a block in a set corresponds to the requested address.
- **Index:** Used to determine the specific set in the cache (not present in fully associative).
- **Offset:** Used to locate the exact word within the block.

In a cache with 2^n sets and block size of 2^m bytes, the address is divided as:
[Tag | Index (n bits) | Offset (m bits)]

e.g.

Simulator:

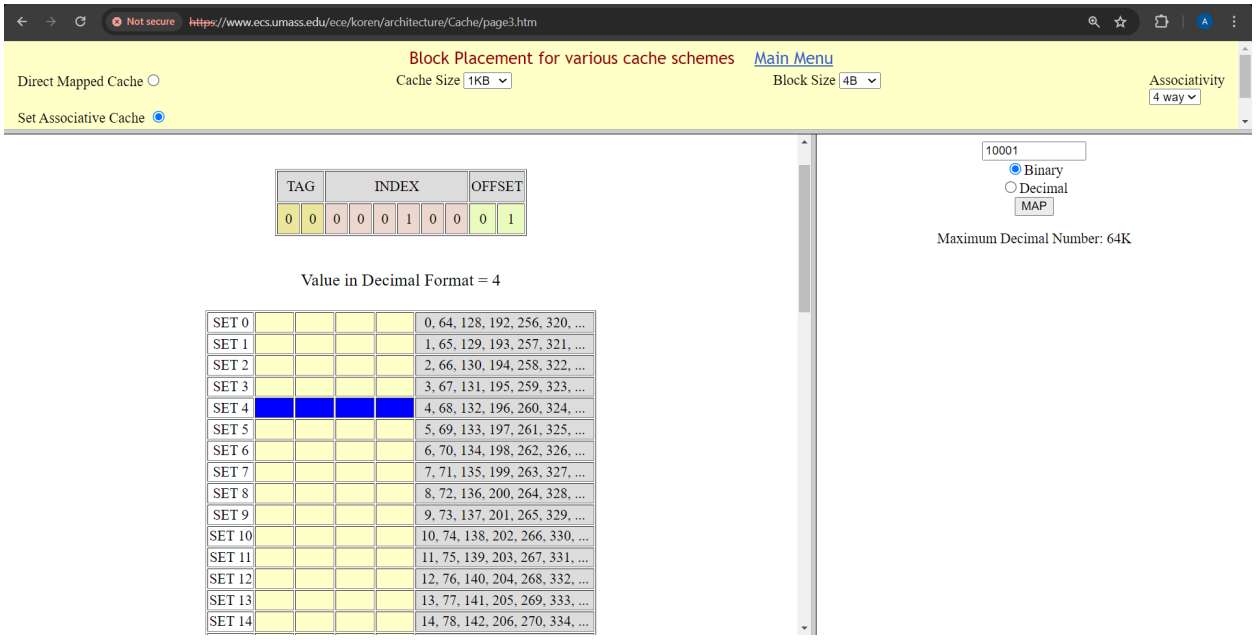


Block Placement

Block placement refers to the rule that determines where a block from main memory will go in the cache:

e.g.

Simulator:



2) Associativity Demonstration - Direct Mapped Caches

In a direct mapped cache, each memory block maps to exactly one cache line determined by the index.
This simplicity makes it fast and easy to implement, but it may suffer from **conflict misses** if multiple frequently used blocks map to the same cache line.

Address Bit Partitioning

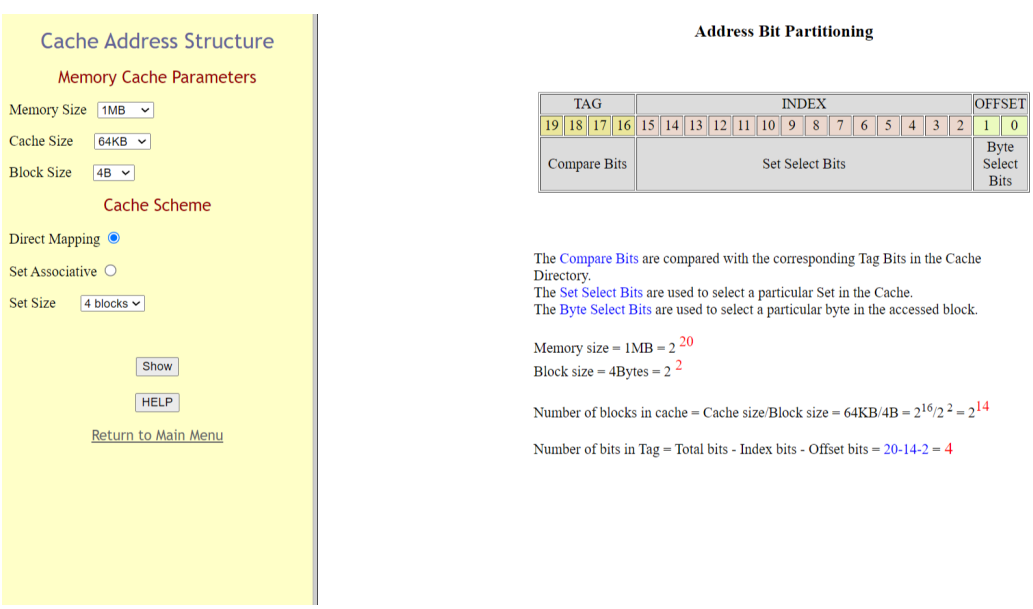
To access data from a direct mapped cache, the memory address is divided into three parts:

- **Tag**: Identifies whether the block currently in the cache line corresponds to the requested memory address.
- **Index**: Directly selects the specific cache line where the block should be stored (only one possible location per block).
- **Offset**: Locates the exact word or byte within the block.

In a cache with 2^n sets and block size of 2^m bytes, the address is divided as:
[Tag | Index (n bits) | Offset (m bits)]

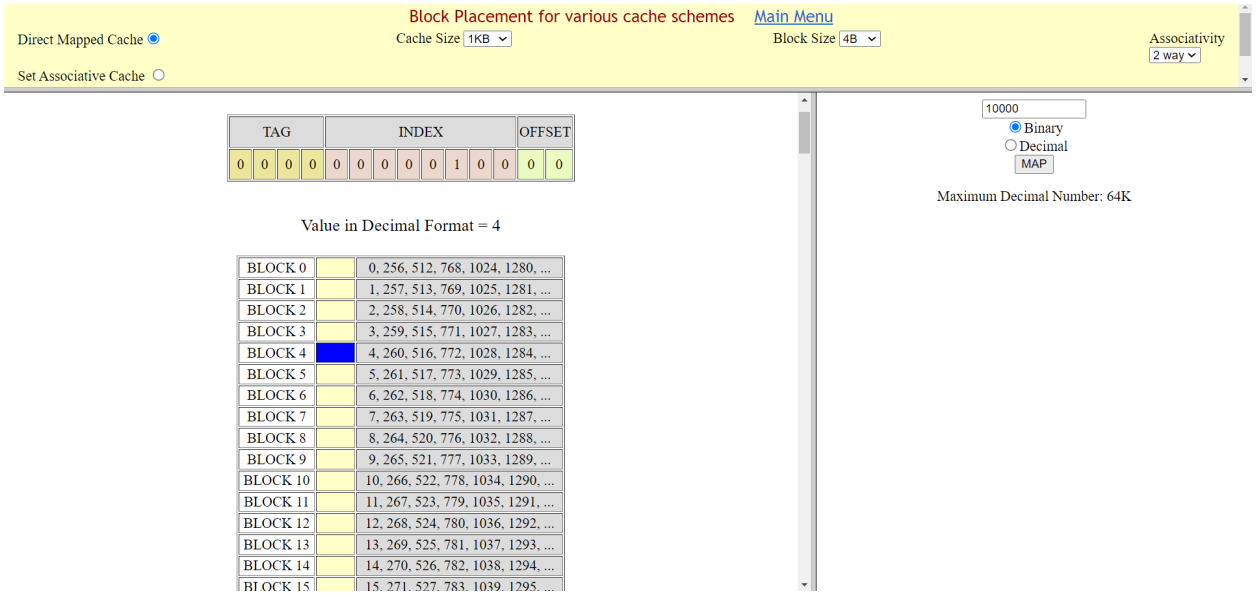
e.g.

Simulator:



Block Placement

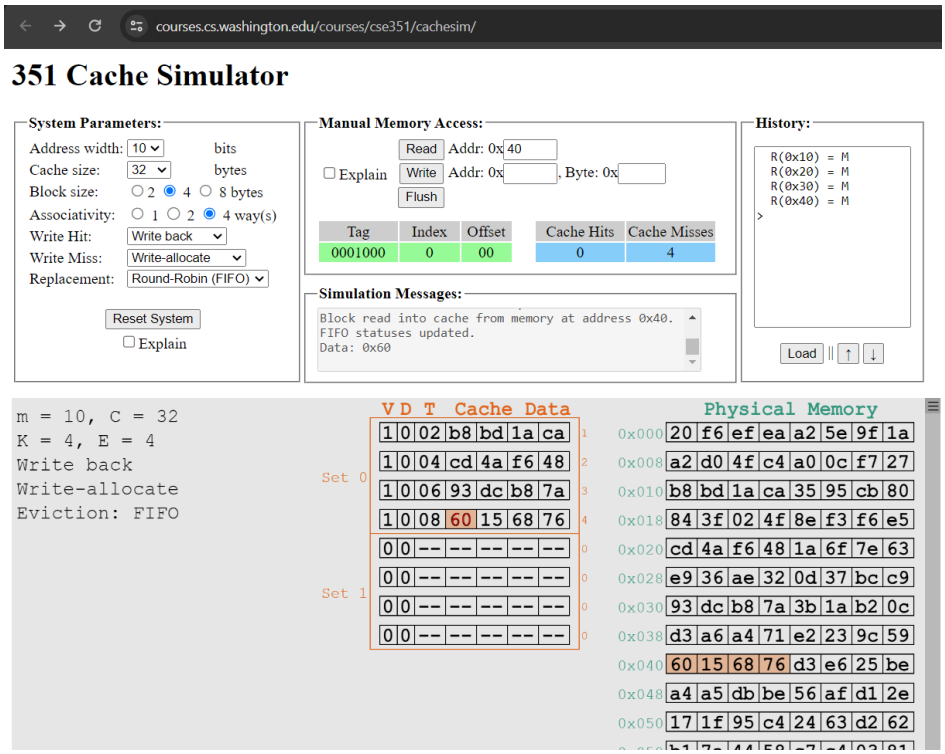
Simulator:



3) Block Replacement Policy - FIFO

FIFO keeps track of the order in which blocks are loaded into the cache and always removes the one that came in first.

Simulator:



courses.cs.washington.edu/courses/cse351/cachesim/

351 Cache Simulator

System Parameters:

Address width: 10 bits
Cache size: 32 bytes
Block size: 2 4 8 bytes
Associativity: 1 2 4 way(s)
Write Hit: Write back
Write Miss: Write-allocate
Replacement: Round-Robin (FIFO)

Reset System
Explain

Manual Memory Access:

Explain
Read Addr: 0x 50
Write Addr: 0x , Byte: 0x
Flush

Tag	Index	Offset	Cache Hits	Cache Misses
0001010	0	00	0	5

Simulation Messages:

```
Block read into cache from memory at address 0x50.
FIFO statuses updated.
Data: 0x17
```

History:

```
R(0x10) = M
R(0x20) = M
R(0x30) = M
R(0x40) = M
R(0x50) = M
>
```

Load

```
m = 10, C = 32
K = 4, E = 4
Write back
Write-allocate
Eviction: FIFO
```

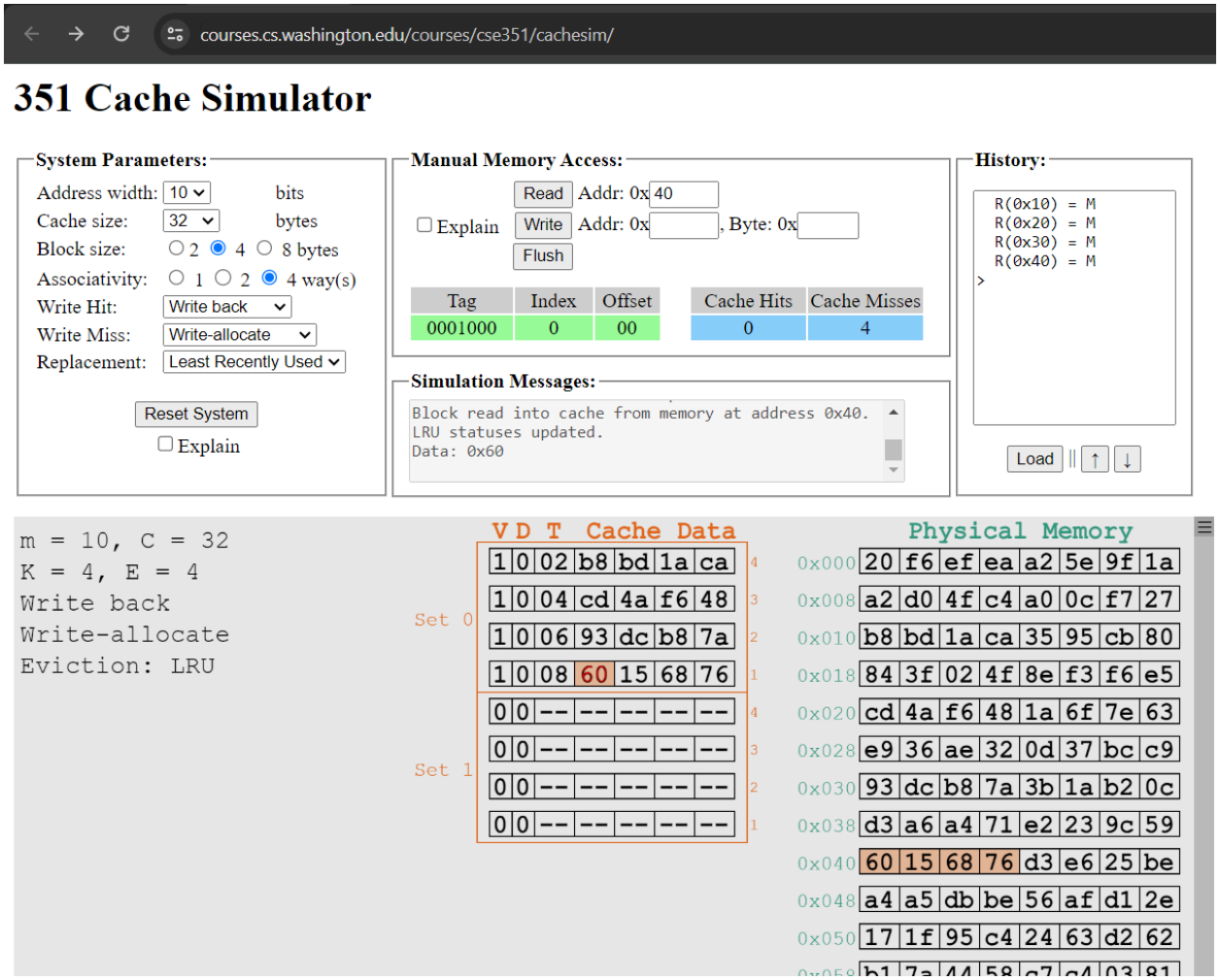
	V	D	T	Cache Data					
Set 0	1	0	0a	17	1f	95	c4	4	
	1	0	04	cd	4a	f6	48	1	
	1	0	06	93	dc	b8	7a	2	
	1	0	08	60	15	68	76	3	
Set 1	0	0	--	--	--	--	--	0	
	0	0	--	--	--	--	--	0	
	0	0	--	--	--	--	--	0	
	0	0	--	--	--	--	--	0	

Physical Memory								
0x000	20	f6	ef	ea	a2	5e	9f	1a
0x008	a2	d0	4f	c4	a0	0c	f7	27
0x010	b8	bd	1a	ca	35	95	cb	80
0x018	84	3f	02	4f	8e	f3	f6	e5
0x020	cd	4a	f6	48	1a	6f	7e	63
0x028	e9	36	ae	32	0d	37	bc	c9
0x030	93	dc	b8	7a	3b	1a	b2	0c
0x038	d3	a6	a4	71	e2	23	9c	59
0x040	60	15	68	76	d3	e6	25	be
0x048	a4	a5	db	be	56	af	d1	2e
0x050	17	1f	95	c4	24	63	d2	62

4) Block Replacement Policy - LRU

LRU (Least Recently Used) evicts the block that has not been used for the longest time.

Simulator:



courses.cs.washington.edu/courses/cse351/cachesim/

351 Cache Simulator

System Parameters:

Address width: 10 bits
Cache size: 32 bytes
Block size: ○ 2 ◉ 4 ○ 8 bytes
Associativity: ○ 1 ○ 2 ◉ 4 way(s)
Write Hit: Write back
Write Miss: Write-allocate
Replacement: Least Recently Used

Reset System
☐ Explain

Manual Memory Access:

☐ Explain
Read Addr: 0x
Write Addr: 0x , Byte: 0x
Flush

Tag	Index	Offset	Cache Hits	Cache Misses
0000010	0	00	1	4

Simulation Messages:

```
Looking for Tag 02... HIT in Line 0!
LRU statuses updated.
Data: 0xb8
```

History:

```
R(0x10) = M
R(0x20) = M
R(0x30) = M
R(0x40) = M
R(0x10) = H
>
```

Load ↑ ↓

```
m = 10, C = 32
K = 4, E = 4
Write back
Write-allocate
Eviction: LRU
```

	V	D	T	Cache Data						
Set 0	1	0	02	b8	bd	1a	ca	1
	1	0	04	cd	4a	f6	48	4
	1	0	06	93	dc	b8	7a	3
	1	0	08	60	15	68	76	2
Set 1	0	0	--	--	--	--	--	4
	0	0	--	--	--	--	--	3
	0	0	--	--	--	--	--	2
	0	0	--	--	--	--	--	1

Physical Memory

0x000	20	f6	ef	ea	a2	5e	9f	1a
0x008	a2	d0	4f	c4	a0	0c	f7	27
0x010	b8	bd	1a	ca	35	95	cb	80
0x018	84	3f	02	4f	8e	f3	f6	e5
0x020	cd	4a	f6	48	1a	6f	7e	63
0x028	e9	36	ae	32	0d	37	bc	c9
0x030	93	dc	b8	7a	3b	1a	b2	0c
0x038	d3	a6	a4	71	e2	23	9c	59
0x040	60	15	68	76	d3	e6	25	be
0x048	a4	a5	db	be	56	af	d1	2e
0x050	17	1f	95	c4	24	63	d2	62

courses.cs.washington.edu/courses/cse351/cachesim/

351 Cache Simulator

System Parameters:

Address width: 10 bits
Cache size: 32 bytes
Block size: ○ 2 ◉ 4 ○ 8 bytes
Associativity: ○ 1 ○ 2 ◉ 4 way(s)
Write Hit: Write back
Write Miss: Write-allocate
Replacement: Least Recently Used

Reset System
☐ Explain

Manual Memory Access:

☐ Explain
Read Addr: 0x 50
Write Addr: 0x , Byte: 0x
Flush

Tag	Index	Offset	Cache Hits	Cache Misses
0001010	0	00	1	5

Simulation Messages:

```
Block read into cache from memory at address 0x50.
LRU statuses updated.
Data: 0x17
```

History:

```
R(0x10) = M
R(0x20) = M
R(0x30) = M
R(0x40) = M
R(0x10) = H
R(0x50) = M
>
```

Load ↑ ↓

```
m = 10, C = 32
K = 4, E = 4
Write back
Write-allocate
Eviction: LRU
```

	V	D	T	Cache Data						
Set 0	1	0	02	b8	bd	1a	ca	2
	1	0	0a	17	1f	95	c4	1
	1	0	06	93	dc	b8	7a	4
	1	0	08	60	15	68	76	3
Set 1	0	0	--	--	--	--	--	4
	0	0	--	--	--	--	--	3
	0	0	--	--	--	--	--	2
	0	0	--	--	--	--	--	1

Physical Memory

0x000	20	f6	ef	ea	a2	5e	9f	1a
0x008	a2	d0	4f	c4	a0	0c	f7	27
0x010	b8	bd	1a	ca	35	95	cb	80
0x018	84	3f	02	4f	8e	f3	f6	e5
0x020	cd	4a	f6	48	1a	6f	7e	63
0x028	e9	36	ae	32	0d	37	bc	c9
0x030	93	dc	b8	7a	3b	1a	b2	0c
0x038	d3	a6	a4	71	e2	23	9c	59
0x040	60	15	68	76	d3	e6	25	be
0x048	a4	a5	db	be	56	af	d1	2e
0x050	17	1f	95	c4	24	63	d2	62
0x058	b1	7a	44	58	c7	c4	03	81

CONCLUSION:

1) This assignment provided a comprehensive understanding of cache memory, covering its levels, mapping and the critical role it plays in bridging the speed gap between the CPU and main memory.
2) Through simulator-based demonstrations, we gained practical insight into how address bit partitioning determines block location.
3) Additionally, we explored two key block replacement policies, including FIFO and LRU, and observed their behavior through simulations.
4) Together, these concepts underline the importance of efficient cache design in optimizing system performance.

REFERENCES:

1. Computer architecture educational tools | Prof. Emeritus Israel Koren, Department of Electrical and Computer Engineering, University of Massachusetts
   https://www.ecs.umass.edu/ece/koren/#simulators

   The following simulators were used:
   a) Cache Address Bit Partitioning:
   https://www.ecs.umass.edu/ece/koren/architecture/Cache/default.htm
   b) Block Placement Demo:
   https://www.ecs.umass.edu/ece/koren/architecture/Cache/page3.htm

2. Cache Simulator | Paul G. Allen School of Computer Science and Engineering, University of Washington
   https://courses.cs.washington.edu/courses/cse351/cachesim/